

RELATÓRIO LLM

Ferramenta utilizada: ChatGPT

Data: 09/06/2026

Versão: GPT-5.5 Gratuita

Acesso à internet: Sim.

1. Explicação conceitual

O conceito usado de teste foi o modelo de Von Neumann. Quando feito a pergunta genérica, a LLM explicou destrinchando todos os detalhes pequenos, passando a impressão de assumir que quem perguntou era um completo leigo, sem mostrar muitos exemplos práticos.

O prompt específico foi: *Explique o modelo de Von Neumann para um aluno de Ciência da Computação, que está estudando para uma prova, de forma simples e direta.*

Dado esse novo prompt, a LLM explicou assumindo que o usuário tem alguma base de conhecimento do assunto, apresentou mais exemplos prático, inclusive demonstrações com código em C.

2. Geração de Código

Primeiro prompt: *Gere um programa que gere uma matriz 3x3 aleatória, e então calcula sua determinante.*

Código gerado:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {

int A[3][3];
srand(time(NULL));
```

```

// Gera a matriz aleatória
printf("Matriz:\n");
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        A[i][j] = rand() % 10;
        printf("%3d ", A[i][j]);
    }
    printf("\n");
}

// Determinante de matriz 3x3
int det =
    A[0][0] * (A[1][1] * A[2][2] - A[1][2] * A[2][1])
    - A[0][1] * (A[1][0] * A[2][2] - A[1][2] * A[2][0])
    + A[0][2] * (A[1][0] * A[2][1] - A[1][1] * A[2][0]);

printf("\nDeterminante = %d\n", det);

return 0;
}

```

Exemplos de execução:

Matriz:

```

0 4 2
3 3 6
2 8 6

```

Determinante = 12

Process returned 0 (0x0) execution time: 0.043 s

Matriz:

```

7 7 4
1 5 2
3 8 7

```

Determinante = 98

Process returned 0 (0x0) execution time: 0.038 s

Análise: a LLM gerou um código em C, com poucos comentários, e assumiu que o usuário iria querer uma representação visual do que o programa faz, ainda que não solicitado.

Prompt melhorado: *Gere um programa em python que gere uma matriz 3x3 aleatória, e então calcula sua determinante, com comentários detalhados sobre cada passo, com exemplos de saída.*

```
# =====
# GERAÇÃO DE UMA MATRIZ 3x3 ALEATÓRIA E CÁLCULO DO
# DETERMINANTE
# =====
#
# Este programa:
# 1. Gera uma matriz 3x3 com números inteiros aleatórios.
# 2. Exibe a matriz gerada.
# 3. Calcula seu determinante utilizando a fórmula para
# matrizes 3x3.
# 4. Exibe o resultado.
#
# Autor: Exemplo educacional
# =====

# Importa o módulo random, responsável por gerar números aleatórios
import random

# -----
# ETAPA 1: GERAR A MATRIZ
# -----
#
# Vamos criar uma lista de listas:
#
# [
# [a11, a12, a13],
# [a21, a22, a23],
# [a31, a32, a33]
# ]
#
# Cada elemento será um inteiro aleatório entre 0 e 9.
#

matriz = []

# Percorre as 3 linhas da matriz
for i in range(3):

    linha = []

    # Gera os 3 elementos da linha atual
    for j in range(3):
        numero = random.randint(0, 9)
        linha.append(numero)
```

```

# Adiciona a linha completa à matriz
matriz.append(linha)

# -----
# ETAPA 2: EXIBIR A MATRIZ
# -----

print("Matriz gerada:")

for linha in matriz:
    print(linha)

# -----
# ETAPA 3: EXTRAIR OS ELEMENTOS
# -----
#
# Para facilitar a leitura da fórmula do determinante,
# armazenamos cada posição em uma variável.
#
# | a b c |
# A = | d e f |
# | g h i |
#
a = matriz[0][0]
b = matriz[0][1]
c = matriz[0][2]

d = matriz[1][0]
e = matriz[1][1]
f = matriz[1][2]

g = matriz[2][0]
h = matriz[2][1]
i = matriz[2][2]

# -----
# ETAPA 4: CALCULAR O DETERMINANTE
# -----
#
# Fórmula:
#
# det(A) =
# a(ei - fh)
# - b(di - fg)
# + c(dh - eg)
#
# Esta é a expansão por cofatores da primeira linha.
#

```

```
determinante = (  
    a * (e * i - f * h)  
    - b * (d * i - f * g)  
    + c * (d * h - e * g)  
)  
  
# -----  
# ETAPA 5: EXIBIR O RESULTADO  
# -----  
  
print("\nDeterminante =", determinante)
```

Exemplos gerados pela própria LLM:

Matriz gerada:

```
[4, 1, 8]  
[2, 5, 7]  
[3, 6, 9]
```

Determinante = 9

Matriz gerada:

```
[1, 2, 3]  
[4, 5, 6]  
[7, 8, 9]
```

Determinante = 0

Comparação: O novo código foi gerado na linguagem desejada (Python), com muito mais comentários e detalhes, além de exemplos de saída desejados.

3. Análise Crítica

Usando o primeiro prompt de geração de código:

1. A resposta estava correta?

Parcialmente. Gerou o código desejado corretamente, mas assumiu que o usuário desejava coisas não explicitamente solicitadas, sem perguntar antes.

2. Havia informações incompletas ou incorretas?

Nesse exemplo, não.

3. Como a resposta foi verificada?

A partir de minha análise crítica e testes de execução do programa.

4. A LLM demonstrou excesso de confiança?

De certa forma sim, ao assumir coisas não solicitadas.

5. Como o prompt poderia ser melhorado?

Com maior especificidade do que se desejada, refinando a resposta da LLM.

4. Boas práticas de uso

1. Por que não devemos copiar respostas sem revisão?

Pois a LLM está sempre sujeita a erros, respostas incompletas ou até resposta levemente equivocadas devido a alguma falta de especificidade do prompt.

2. Por que é importante informar os prompts utilizados?

Para sabermos a partir de que a LLM gerou aquela resposta, ajudando a corrigir o prompt, ver se a LLM errou de fato, verificando se ela gerou informações desnecessárias etc.

3. Em quais situações as LLMs ajudam no aprendizado?

A LLM é boa para responder questões específicas, sendo mais direta. Pode ser bastante útil para estudantes, mas tudo que for gerado deve ser verificado.

4. Em quais situações elas podem atrapalhar?

Quando utilizadas, por exemplo, para gerar código de maneira autônoma e com pouca verificação.

5. O que significa utilizar uma LLM como apoio e não como substituta do raciocínio?

Significa que não devemos confiar cegamente nas respostas geradas, sendo necessária sempre a verificação da resposta por alguma humano.

5. Reflexão sobre o uso de IA Generativa

O uso de IA generativa pode ser muito útil, mas deve ser usado com cautela, sempre verificando suas respostas e atentando-se ao prompt utilizado, uma vez que ele pode influenciar fortemente o resultado obtido. A ferramenta utilizada foi bastante tanto para geração de código, quando para explicações conceituais, mas precisou de verificação sobre suas respostas, já que frequentemente assumia informações não especificadas, especialmente na geração de código.

Eu utilizaria essa ferramenta, mas com pouca frequência e, sempre que o fizesse, tomaria cuidado com todos os prompt dados e verificaria todas as respostas geradas.